

Technical Specification Document

1. Introduction

- **Project Title:** Hawk-Eye: 3D structure from motion solution, spatial segmentation, classification
- **Document Version:** 1.0
- **Date:** 15/11/2024
- **Prepared by:** Fady Damak

2. Version history

#	Ver.	Date	Author	Description
00	0.0	2024.07.08.	T. Storcz	Template generation
01	0.0	2024.09.25	Fady Damak	Kick-off document creation
02	0.0	2024.10.09	Fady Damak	Functional Specification
03	0.0	2024.11.13	Fady Damak	Technical Specification

3. Purpose

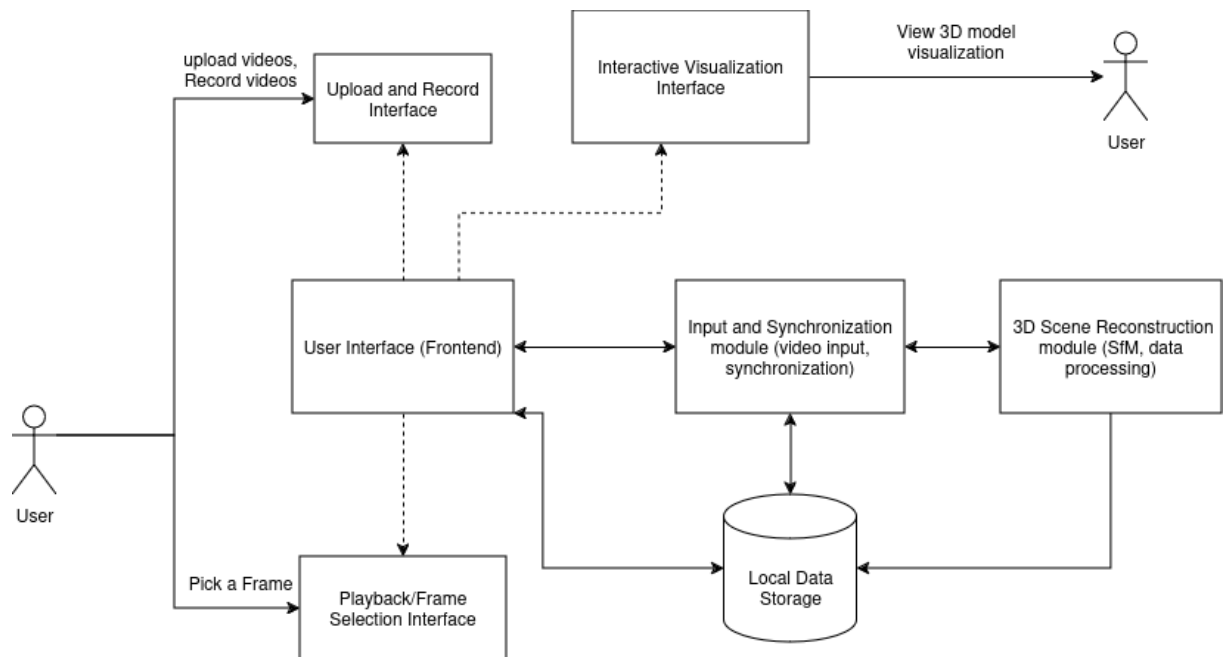
- **Purpose Statement:** The purpose of this document is to outline the technical details, architecture, and requirements for the Hawk-Eye system, focusing on its ability to dynamically generate 3D scene reconstructions based on synchronized video inputs. It aims to serve as a reference for system development, testing, and deployment.
- **Scope:** This document covers the system's architecture, data flow, components, interfaces, and non-functional requirements. It excludes detailed user manuals or operational guides.

4. System Overview

- **System Description:**The Hawk-Eye system enables users to record synchronized video footage from multiple cameras, process the frames to reconstruct a 3D model of the volleyball court and ball at specific timestamps, and view the results interactively. This system aids in post-game analysis and decision-making.
- **Architecture Overview:** The system consists of three core components:
 1. **Input and Synchronization:** Handles video uploads or synchronized live recording from multiple cameras.
 2. **3D Scene Reconstruction:** Processes the video frames to generate a 3D model of the selected scene using structure-from-motion techniques.
 3. **Interactive Visualization:** Provides users with an interface to explore the 3D model, allowing them to rotate, zoom, and inspect the reconstructed scene.

5. System Architecture

- **High-Level Architecture Diagram:**



- **Component Descriptions:**

User Interface (UI):

- **Responsibilities:** Accepts input for uploading videos, guides users in recording synchronized footage, allows playback for frame selection, and shows the 3D reconstruction.

Input & Synchronization:

- **Responsibilities:** Handles video upload/recording, and synchronizes multiple camera feeds. This may also include synchronizing the video timestamps for accurate frame matching.

3D Reconstruction:

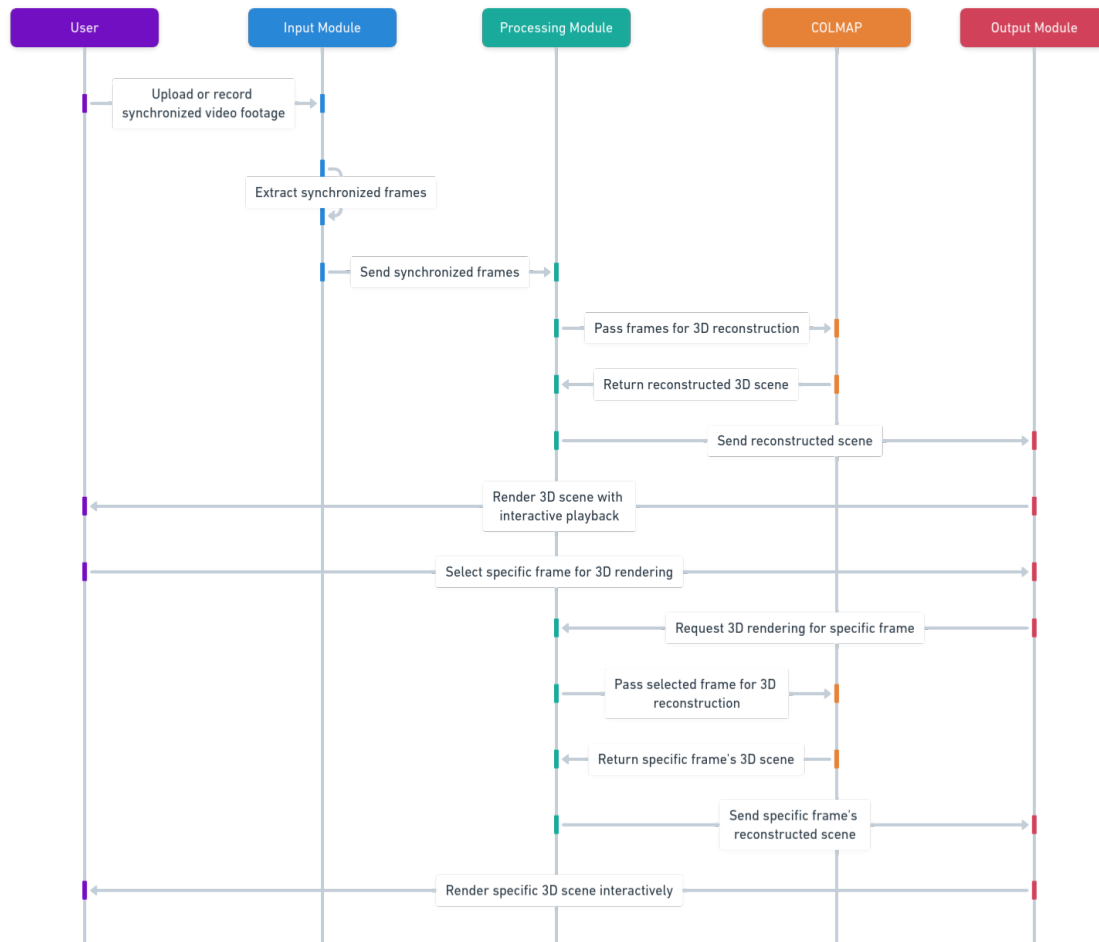
- **Responsibilities:** Processes synchronized video frames using Structure from Motion (SfM) techniques to generate a 3D model of the court, ball, and other relevant elements.

Interactive Visualization:

- **Responsibilities:** Renders the 3D model and provides controls for the user to interact with the model (e.g., zoom, rotate, select frames).
- **Interactions:** Receives the 3D model from **3D Reconstruction** and displays it to the user.

- **Component integration:**

This sequence diagram shows the data flow in the system



Input Module to Processing Module:

- Data Flow: The Input Module collects video frames from multiple cameras or uploaded files and ensures they are synchronized.
- Communication: Frames are temporarily stored in memory or local storage and passed to the Processing Module for analysis.

Processing Module to Output Module:

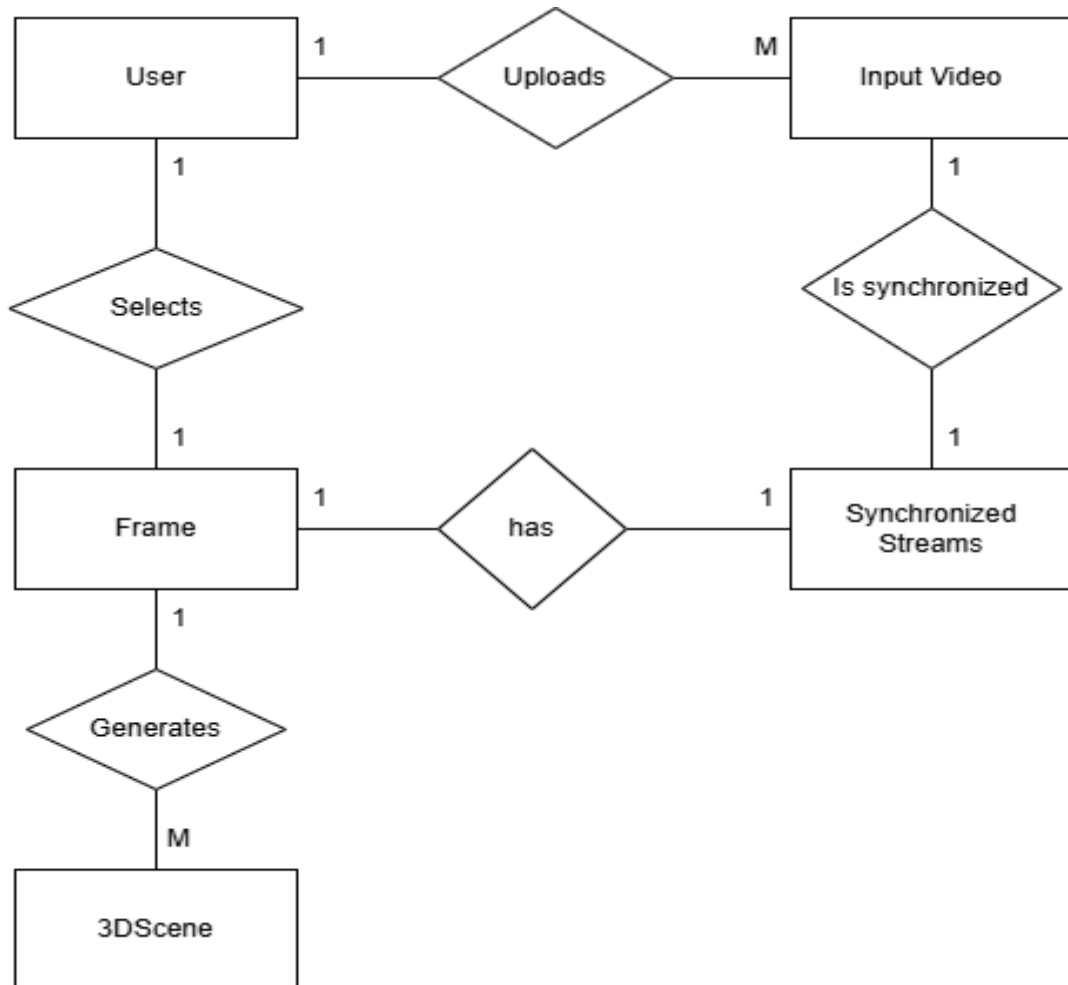
- Data Flow: The Processing Module generates a 3D scene from the synchronized frames and sends the result to the Output Module.
- Communication: Results are stored temporarily in memory and rendered in the Output Module using WebGL or PyQt5's rendering engine.

Integration with External Libraries:

- COLMAP is invoked by the Processing Module for 3D reconstruction.
- FFmpeg ensures video compatibility and preprocessing.
- PyQt5 handles UI interactions and rendering output.

6. Data Design

- **Data Model:**



- **Database Schema:** (No persistent database is required as per project scope.)

7. Interface Specifications

- **User Interface (UI):** Figma designs available in this [link](#)
 - **Upload/Record Page:** Users can upload videos or initiate synchronized recording sessions.
 - **Processing Status Page:** Displays progress and completion status for the 3D reconstruction process.

- **Results Viewer:** Allows users to view and interact with the generated 3D models, including features like zoom, rotation, and playback.
 - **Settings Page:** Includes options for camera setup and system configuration.
-
- **Application Programming Interface (API):**

Purpose:

The API serves as the backbone for the system's functionality, enabling seamless interaction between the user interface, processing modules, and storage mechanisms. It ensures structured communication, allowing users to upload video files, trigger 3D reconstruction and ball-tracking processes, and retrieve results efficiently.

Key Features:

1. **Video Upload:**
 - Allows users to upload video footage for analysis.
 - Validates file format, resolution, and metadata before processing.
2. **Processing Task Management:**
 - Initiates 3D reconstruction and ball-tracking computations.
 - Tracks and manages the status of ongoing processing tasks.
3. **Status Monitoring:**
 - Provides updates on the progress of processing tasks (e.g., queued, in-progress, completed).
4. **Result Retrieval:**
 - Delivers processed outputs, including the 3D model of the ball's trajectory and any accompanying data (e.g., trajectory coordinates).

Authentication:

- APIs will primarily operate within the desktop environment, making user authentication minimal. However, optional external integrations (e.g., cloud data storage or analytics) may require secure authentication methods like API keys or token-based authentication.

Scalability:

The APIs are designed to handle:

- Multiple video uploads simultaneously (scalable within the hardware limits).
- Dynamic configuration to adjust for varying input resolutions or camera setups.

Error Handling:

- Comprehensive error messages will be provided for issues like invalid file formats, processing failures, or hardware constraints.
- Logs will be maintained to troubleshoot and refine performance.

- **External Interfaces:**

- **COLMAP:** Used for structure-from-motion (SfM) reconstruction. It receives synchronized image frames as input and generates a 3D model of the ball's trajectory.
- **OpenCV:** For frame extraction and synchronization.
- **WebGL or PyQt5 3D Modules:** Used to render the 3D model interactively within the desktop application.
- **FFmpeg:** Handles video preprocessing, ensuring all uploaded or recorded videos meet the required formats and quality standards.
- **Camera Modules:** External HD cameras with compatible drivers and protocols for real-time synchronization and video capture.

8. Non-Functional Requirements

- **Performance:** The generation of a 3D scene for each frame shouldn't exceed a couple of minutes and that depends on the cameras' resolution, processing power and number of cameras. During implementation we will test out how many cameras the system can handle starting with 2 cameras for depth perception.
- **Security:**
 - **Authentication:** Only authenticated users can upload videos or access results.
 - **Data Protection:** Secure uploaded videos and processing data using encrypted storage (if data is stored temporarily).
 - **Access Control:** Ensure only authorized users can view or download results.
- **Usability:**
 - Intuitive UI for non-technical users, including coaches and analysts.
 - Clear feedback during video upload, processing, and result visualization.
 - Include tooltips and guides for camera setup.
- **Reliability:**
 - Ensure **99% uptime** during system use.

- Implement error-handling mechanisms to notify users of any issues during processing or synchronization.
- **Scalability:**
 - Design the system to handle additional cameras (maximum number will be determined during implementation) and higher resolution videos in the future.
 - Modular architecture to add features like player tracking or live match analysis later.
- **Compliance:**
 - Allow users to delete their uploaded videos and generated results.

9. Technology Stack

- **Programming Languages:**
 - **Python:** Primary language for the backend logic, video processing, and 3D reconstruction tasks.
 - **C++:** For high-performance tasks where Python may fall short (e.g., integrating with COLMAP or OpenMVG).
 - **JavaScript:** If WebGL is used for rendering the 3D model within the desktop app interface.

- **Frameworks and Libraries:**

Backend and Core Logic:

- **COLMAP:** For 3D structure-from-motion reconstruction from synchronized video frames.
- **OpenCV:** For video synchronization, frame extraction, and preprocessing tasks.
- **NumPy** and **SciPy:** For numerical computations during trajectory analysis.

Desktop Application Framework:

- **PyQt5 or Electron:** For creating the GUI (graphical user interface) of the desktop app.

3D Rendering:

- **OpenGL:** For rendering the 3D scene directly within the app.

- **Three.js (with Node.js backend):** If you plan to embed a lightweight web-based rendering engine within the app.

Video Handling:

- **FFmpeg:** For video transcoding and format compatibility checks.
- **H.264 codec:** Ensures compatibility with most HD video formats.

- **Development Tools:**

IDE:

- PyCharm or Visual Studio Code for Python development.
- Qt Creator for PyQt5 or PySide2 UI design.

Version Control:

- Git for source code management and collaboration. Github for remote version control

Build Tools:

- PyInstaller: For bundling the Python application into standalone executables for deployment.
- CMake: For managing C++ build processes, especially for COLMAP integration.

- **Deployment Environment:**

Local Machines:

Hardware Requirements:

- CPU: Multi-core processor for video processing tasks.
- GPU: Required for 3D reconstruction and rendering (NVIDIA or AMD recommended with CUDA/OpenCL support).
- RAM: Minimum 8GB, recommended 16GB or higher for smoother processing.
- Storage: At least 5GB free space for temporary video files and outputs.

- **Operating Systems Supported:**

- Windows 10/11: Primary platform for most users.